WinDivert фильтры (`--wf-*`) в zapret2 для Windows

WinDivert — это драйвер для Windows, который позволяет перехватывать и модифицировать сетевые пакеты на уровне ядра. В zapret2 для Windows используется программа **winws2** (вместо nfqws2 для Linux), которая работает с WinDivert.

WinDivert фильтр применяется ПЕРВЫМ — это фундаментальная архитектура.

Порядок фильтрации (pipeline):

```
┌──────────────────────────────────────────────────────────────────┐
│                          СЕТЕВОЙ СТЕК                            │
│                        (все пакеты)                              │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                1  WinDivert ФИЛЬТР (--wf-*)                      │
│                      [уровень драйвера]                          │
│                                                                  │
│   Пропускает только пакеты, соответствующие фильтру              │
│   Остальные идут мимо nfqws напрямую в сеть                      │
└──────────────────────────────────────────────────────────────────┘
                              │ только захваченные пакеты
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                        nfqws                                     │
│                                                                  │
│   2  --filter-tcp/udp   - фильтр по портам                       │
│   3  --ipset            - фильтр по IP                           │
│   4  --hostlist         - фильтр по доменам (из payload)         │
│   5  desync actions     - применение обхода DPI                  │
└──────────────────────────────────────────────────────────────────┘
```

Почему это важно:

- **Производительность**: WinDivert фильтрует на уровне драйвера — CPU-efficient
- **nfqws видит только нужные пакеты** — не тратит ресурсы на разбор всего трафика
- `--wf-*` = **грубый фильтр** (порты, протоколы, IP версия)
- `--hostlist` = **тонкий фильтр** (по содержимому пакета, доменам)

Да, `--wf-*` применяется первым, на уровне драйвера, до того как пакет попадёт в nfqws.

📦 Основные параметры WinDivert фильтров

1. `--wf-iface` - Фильтр по сетевому интерфейсу

Синтаксис:

```
--wf-iface=<int>[.<int>]
```

Параметры:

- `int` - индекс сетевого интерфейса
- `int.int` - индекс интерфейса и подинтерфейса

Примеры:

```
--wf-iface=1        # интерфейс 1
--wf-iface=2.0      # интерфейс 2, подинтерфейс 0
```

---

2. `--wf-l3` - Фильтр L3 протокола

Синтаксис:

```
--wf-l3=ipv4|ipv6
```

Параметры:

- `ipv4` - только IPv4
- `ipv6` - только IPv6
- Можно указывать несколько через запятую

Примеры:

```
--wf-l3=ipv4
--wf-l3=ipv4,ipv6
```

---

3. `--wf-tcp-in` - TCP входящие порты

Синтаксис:

```
--wf-tcp-in=[~]port1[-port2]
```

Параметры:

- `port` - конкретный порт
- `port1-port2` - диапазон портов
- `~port` - отрицание (все кроме)
- Можно указывать несколько через запятую

Примеры:

```
--wf-tcp-in=80,443        # входящие на порты 80 и 443
--wf-tcp-in=1000-2000     # диапазон
--wf-tcp-in=~443          # все кроме 443
```

---

4. `--wf-tcp-out` - TCP исходящие порты

Синтаксис:

```
--wf-tcp-out=[~]port1[-port2]
```

Параметры: (аналогично `--wf-tcp-in`)

Примеры:

```
--wf-tcp-out=80,443      # исходящие на порты 80 и 443
```

---

5. `--wf-udp-in` - UDP входящие порты

Синтаксис:

```
--wf-udp-in=[~]port1[-port2]
```

Параметры: (аналогично TCP)

Примеры:

```
--wf-udp-in=53,443       # DNS и QUIC входящие
```

---

6. `--wf-udp-out` - UDP исходящие порты

Синтаксис:

```
--wf-udp-out=[~]port1[-port2]
```

Параметры: (аналогично TCP)

Примеры:

```
--wf-udp-out=443         # QUIC исходящие
```

---

7. `--wf-tcp-empty` - Обработка пустых TCP пакетов

Синтаксис:

```
--wf-tcp-empty=[0|1]
```

Параметры:

- `0` - не обрабатывать пустые TCP пакеты без флагов SYN/RST/FIN (по умолчанию)
- `1` - обрабатывать пустые TCP пакеты

Примеры:

```
--wf-tcp-empty=1          # включить обработку пустых пакетов
```

---

8. `--wf-raw-part` - Частичный raw фильтр ⭐

Синтаксис:

```
--wf-raw-part=<filter>|@<filename>
```

Параметры:

- `filter` - строка фильтра на языке WinDivert
- `@filename` - загрузить фильтр из файла
- Можно указывать **несколько раз** - они объединяются через OR

Особенности:

- Фильтры работают на уровне ядра (очень быстро)
- Язык фильтров похож на tcpdump/Wireshark
- Позволяет фильтровать по содержимому payload
- Экономит CPU, не перенаправляя весь поток

Примеры:

```
# Прямо в командной строке
--wf-raw-part="outbound and udp.DstPort=443 and udp.PayloadLength>=256"

# Из файла
--wf-raw-part=@windivert_part.discord_media.txt

# Несколько фильтров
--wf-raw-part=@discord.txt --wf-raw-part=@stun.txt --wf-raw-part=@quic.txt
```

---

9. `--wf-filter-lan` - Фильтр локальных сетей

Синтаксис:

```
--wf-filter-lan=0|1
```

Параметры:

- `0` - не фильтровать локальные сети
- `1` - исключить локальные IP адреса (по умолчанию)

Исключаемые диапазоны:

- IPv4: 127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16
- IPv6: ::1/128, fc00::/7, fe80::/10, ff00::/8

Примеры:

```
--wf-filter-lan=0        # не исключать локальные сети
```

---

10. `--wf-raw` - Полный raw фильтр

Синтаксис:

```
--wf-raw=<filter>|@<filename>
```

Параметры:

- `filter` - полная строка фильтра WinDivert
- `@filename` - загрузить из файла
- **ЗАМЕНЯЕТ** все `--wf-tcp`, `--wf-udp`, `--wf-raw-part`

Примеры:

```
--wf-raw="!impostor and !loopback and outbound and tcp.DstPort=443"
--wf-raw=@full_filter.txt
```

---

11. `--wf-save` - Сохранить фильтр и выйти

Синтаксис:

```
--wf-save=<filename>
```

Параметры:

- `filename` - путь для сохранения сгенерированного фильтра
- После сохранения программа завершается

Примеры:

```
--wf-tcp-out=80,443 --wf-save=generated_filter.txt
```

🔤 Язык фильтров WinDivert

Основные операторы:

Логические:

- and - И
- or - ИЛИ
- not или ! - НЕ

Сравнения:

- == или = - равно
- != - не равно
- <, >, <=, >= - сравнения

Направление:

- outbound - исходящие пакеты
- inbound - входящие пакеты

Специальные:

- impostor - пакеты от других драйверов
- loopback - loopback интерфейс

Доступные поля:

IP (IPv4):

```
ip                      # IPv4 пакет
ip.SrcAddr              # IP источника
ip.DstAddr              # IP назначения
ip.Length               # Длина IP пакета
ip.TTL                  # Time To Live
ip.Protocol             # Протокол (6=TCP, 17=UDP)
```

IPv6:

```
ipv6                    # IPv6 пакет
ipv6.SrcAddr            # IPv6 источника
ipv6.DstAddr            # IPv6 назначения
ipv6.Length             # Длина пакета
ipv6.HopLimit           # Hop Limit
ipv6.NextHdr            # Следующий заголовок
```

TCP:

```
tcp                     # TCP пакет
tcp.SrcPort             # Порт источника
tcp.DstPort             # Порт назначения
tcp.Syn                 # SYN флаг
tcp.Rst                 # RST флаг
tcp.Fin                 # FIN флаг
tcp.Ack                 # ACK флаг
tcp.Psh                 # PSH флаг
tcp.PayloadLength       # Длина payload
tcp.Payload[N]          # Байт payload по индексу N
tcp.Payload16[N]        # 16-bit значение (little-endian)
tcp.Payload32[N]        # 32-bit значение (little-endian)
```

UDP:

```
udp                     # UDP пакет
udp.SrcPort             # Порт источника
udp.DstPort             # Порт назначения
udp.PayloadLength       # Длина payload
udp.Payload[N]          # Байт payload по индексу N
udp.Payload16[N]        # 16-bit значение (little-endian)
udp.Payload32[N]        # 32-bit значение (little-endian)
```

Интерфейс:

```
ifIdx                   # Индекс интерфейса
subIfIdx                # Индекс подинтерфейса
```

📝 Примеры фильтров

Пример 1: QUIC Initial (IETF)

```
outbound and
udp.PayloadLength>=256 and
udp.Payload[0]>=0xC0 and udp.Payload[0]<0xD0 and
udp.Payload[1]=0 and udp.Payload16[1]=0 and udp.Payload[4]=1
```

Пример 2: STUN

```
outbound and
udp.PayloadLength>=20 and
udp.Payload32[1]=0x2112A442 and udp.Payload[0]<0x40
```

Пример 3: WireGuard

```
outbound and
udp.PayloadLength=148 and
udp.Payload[0]=0x01
```

Пример 4: Discord Media

```
outbound and ip and
udp.DstPort>=50000 and udp.DstPort<=50099 and
udp.PayloadLength=74 and
udp.Payload32[0]=0x00010046 and
udp.Payload32[2]=0 and
udp.Payload32[3]=0
```

Пример 5: TLS Client Hello

```
outbound and
tcp.PayloadLength>=5 and
tcp.Payload[0]=0x16 and
tcp.Payload[1]=0x03 and
tcp.Payload[2]>=0x01 and tcp.Payload[2]<=0x03
```

Пример 6: HTTP Redirect (302/307)

```
tcp.PayloadLength>=12 and
tcp.Payload32[0]==0x48545450 and
tcp.Payload16[2]==0x2F31 and
tcp.Payload[6]==0x2E and
tcp.Payload16[4]==0x2033 and
tcp.Payload[10]==0x30 and
(tcp.Payload[11]==0x32 or tcp.Payload[11]==0x37)
```

💡 Полные примеры использования

Пример 1: Базовый перехват HTTP/HTTPS

```
winws2 ^
  --wf-tcp-out=80,443 ^
  --lua-init=@zapret-lib.lua --lua-init=@zapret-antidpi.lua ^
  --filter-tcp=80,443 --filter-l7=http,tls ^
  --lua-desync=fake:blob=fake_default_tls
```

Пример 2: С частичными фильтрами

```
winws2 ^
  --wf-tcp-out=80,443 ^
  --wf-raw-part=@windivert_part.discord_media.txt ^
  --wf-raw-part=@windivert_part.stun.txt ^
  --wf-raw-part=@windivert_part.quic_initial_ietf.txt ^
  --lua-init=@zapret-lib.lua --lua-init=@zapret-antidpi.lua ^
  --filter-tcp=80,443 --filter-l7=http,tls ^
  --lua-desync=fake:blob=fake_default_tls ^
  --new ^
  --filter-l7=quic,stun,discord ^
  --lua-desync=fake:blob=fake_default_quic:repeats=2
```

Пример 3: Только QUIC с фильтром

```
winws2 ^
  --wf-raw-part=@windivert_part.quic_initial_ietf.txt ^
  --lua-init=@zapret-lib.lua --lua-init=@zapret-antidpi.lua ^
  --filter-l7=quic ^
  --lua-desync=fake:blob=fake_default_quic:repeats=6
```

Пример 4: Полный raw фильтр

```
winws2 ^
  --wf-raw="!impostor and !loopback and outbound and tcp.DstPort=443 and
tcp.PayloadLength>0" ^
  --lua-init=@zapret-lib.lua --lua-init=@zapret-antidpi.lua ^
  --filter-tcp=443 ^
  --lua-desync=fake:blob=fake_default_tls
```

⚠️ Важные особенности и ограничения

1. Отсутствие conntrack

- WinDivert НЕ отслеживает соединения (нет аналога connbytes из iptables)
- Если перехватывать порт целиком, весь трафик идет через winws2
- Это нагружает процессор при передаче больших объемов данных

2. Ограничения языка фильтров

- Нет операций с битовыми полями
- Нет сдвигов (shift)
- Нет побитовой логики (AND, OR, XOR)
- Фильтры могут пропускать неправильные пакеты
- Дофильтрация выполняется в winws2

3. Рекомендации

- Используйте `--wf-raw-part` для точной фильтрации payload
- Это экономит CPU, отсекая ненужный трафик на уровне ядра
- Комбинируйте несколько `--wf-raw-part` для разных протоколов

4. Автоматическая генерация

- winws2 автоматически добавляет `!impostor and !loopback`
- Автоматически добавляет фильтр для SYN/RST/FIN (для conntrack)
- Автоматически добавляет фильтр HTTP redirect (302/307)
- Автоматически исключает локальные сети (если `--wf-filter-lan=1`)

📚 Документация WinDivert

Полная документация языка фильтров:
https://reqrypt.org/windivert-doc.html#filter_language

🎯 Резюме

WinDivert фильтры — это мощный механизм для перехвата пакетов в Windows на уровне ядра:

- `--wf-tcp-in/out`, `--wf-udp-in/out` — простая фильтрация по портам
- `--wf-raw-part` — частичные фильтры по содержимому payload (рекомендуется!)
- `--wf-raw` — полный контроль над фильтром
- **Язык фильтров** — похож на tcpdump, но с ограничениями
- **Производительность** — фильтры работают в ядре, очень быстро

Используйте `--wf-raw-part` для оптимальной производительности и точной фильтрации!

Разница между `--wf-tcp-out=80,443` и `--wf-tcp-out=80 --wf-tcp-out=443`

🔍 Анализ кода

Изучив функцию `wf_make_pf` в коде (строки 1193-1222), я могу точно объяснить разницу:

✅ Вариант 1: `--wf-tcp-out=80,443` (ПРАВИЛЬНЫЙ)

Что происходит:

1. Параметр обрабатывается **один раз**
2. Функция `wf_make_pf` парсит строку `"80,443"` и разбивает по запятой
3. Генерируется **один** фильтр WinDivert:

```
((tcp.DstPort == 80) or (tcp.DstPort == 443))
```

Результат:

- ✅ Один вызов парсера
- ✅ Оптимальный фильтр
- ✅ Меньше строковых операций

❌ Вариант 2: `--wf-tcp-out=80 --wf-tcp-out=443` (НЕПРАВИЛЬНЫЙ)

Что происходит:

1. Первый параметр `--wf-tcp-out=80` обрабатывается
2. Генерируется фильтр: `((tcp.DstPort == 80))`
3. **Второй параметр ПЕРЕЗАПИСЫВАЕТ первый!**
4. Генерируется новый фильтр: `((tcp.DstPort == 443))`

Результат:

- ❌ **Первый порт (80) теряется!**
- ❌ Работает только последний указанный порт (443)
- ❌ Два вызова парсера (лишняя работа)

📊 Доказательство из кода

Посмотрим на обработку параметра (строки 2330-2337):

```
case IDX_WF_TCP_OUT:
    hash_wf_tcp_out = hash_jen(optarg, strlen(optarg));
    if (!wf_make_pf(optarg, "tcp", "SrcPort", params.wf_pf_tcp_src_out,
WINDIVERT_PORTFILTER_MAX) ||
        !wf_make_pf(optarg, "tcp", "DstPort", params.wf_pf_tcp_dst_out,
WINDIVERT_PORTFILTER_MAX))
    {
        DLOG_ERR("bad value for --wf-tcp-out\n");
        exit_clean(1);
    }
    break;
```

Ключевой момент: Каждый вызов `--wf-tcp-out` **перезаписывает** буферы `params.wf_pf_tcp_src_out` и `params.wf_pf_tcp_dst_out`, а не добавляет к ним!

🔬 Как работает `wf_make_pf`

```
static bool wf_make_pf(char *opt, const char *l4, const char *portname, char *buf,
size_t len)
{
    // ...
    for (n = 0, p = opt, *buf = '(', buf[1] = 0; p; n++)  // Начинает с '('
    {
        if ((e = strchr(p, ',')))  // Ищет запятую
        {
            c = *e;
            *e = 0;
        }
        if (!pf_parse(p, &pf)) return false;

        // Генерирует условие для порта
        if (pf.from == pf.to)
            snprintf(s1, sizeof(s1), "(%s.%s %s %u)", l4, portname, pf.neg ? "!=" :
"==", pf.from);

        if (n) strncat(buf, " or ", len - strlen(buf) - 1);  // Добавляет "or" между
портами
        strncat(buf, s1, len - strlen(buf) - 1);

        // ...
    }
    strncat(buf, ")", len - strlen(buf) - 1);  // Закрывает скобку
    return true;
}
```

Функция:

1. Парсит строку по запятым
2. Для каждого порта генерирует условие
3. Объединяет их через `or`
4. Возвращает один готовый фильтр

📈 Производительность

Вариант 1: `--wf-tcp-out=80,443`

Время парсинга:

- 1 вызов `wf_make_pf`
- 2 итерации цикла внутри функции
- 1 выделение строки

Итоговый фильтр:

```
((tcp.DstPort == 80) or (tcp.DstPort == 443))
```

Производительность WinDivert: ⚡ Оптимальная

- Один компактный фильтр
- Эффективная проверка в ядре

Вариант 2: `--wf-tcp-out=80 --wf-tcp-out=443`

Время парсинга:

- 2 вызова `wf_make_pf`
- 2 выделения строк
- **Первый результат выбрасывается!**

Итоговый фильтр:

```
((tcp.DstPort == 443))  // Порт 80 потерян!
```

Производительность WinDivert: ❌ Неправильная работа

- Фильтрует только последний порт
- Порт 80 не перехватывается!

🎯 Вывод

✅ Используйте:

```
--wf-tcp-out=80,443
```

Преимущества:

1. ✅ **Работает правильно** - оба порта перехватываются
2. ✅ **Быстрее парсинг** - один вызов вместо двух
3. ✅ **Меньше памяти** - одна строка вместо двух
4. ✅ **Читабельнее** - короче командная строка

❌ НЕ используйте:

```
--wf-tcp-out=80 --wf-tcp-out=443
```

Проблемы:

1. ❌ **Не работает!** - только последний порт (443)
2. ❌ **Медленнее** - лишний вызов функции
3. ❌ **Больше кода** - длиннее командная строка
4. ❌ **Теряются данные** - первый порт (80) игнорируется

📋 Правильные примеры

```
# Правильно: несколько портов через запятую
--wf-tcp-out=80,443
--wf-tcp-out=80,443,8080,8443
--wf-udp-out=53,443,853

# Правильно: диапазоны
--wf-tcp-out=1000-2000
--wf-tcp-out=80,443,8000-9000

# Правильно: отрицание
--wf-tcp-out=~443
--wf-tcp-out=~80,~443

# НЕПРАВИЛЬНО: повторяющиеся параметры
--wf-tcp-out=80 --wf-tcp-out=443  # ❌ Работает только 443!
```

🔧 Техническое объяснение

Параметры командной строки в `getopt_long` обрабатываются **последовательно**. Когда встречается один и тот же параметр дважды, второй вызов **перезаписывает** результат первого, потому что они записываются в **один и тот же буфер** (`params.wf_pf_tcp_dst_out`).

Это не баг, а особенность дизайна - параметры WinDivert фильтров не предназначены для множественного указания. Вместо этого используется **встроенная поддержка списков через запятую**.

Итог: Всегда используйте запятые для перечисления портов в одном параметре. Это правильно, быстрее и работает как задумано!